INTEGRAL DRAGON

MODULE 3 | SESSION 12

OWASP Top 10 for Agentic Apps

The security problem changes once the agent can act.

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

Bad output is not the main risk anymore.

- A hallucination is incorrect content.
- A forged API call is a security incident.
- Once the agent can use tools, the threat model changes.
- The real question becomes: what bad action can the system be manipulated into taking?

Mal-content

Wrong text, weak reasoning, or a bad recommendation.

Mal-action

Unauthorized tool use, privilege abuse, or data exfiltration in the real environment.

Agentic systems create new abuse paths that chatbot-only systems do not.

Tool misuse

The agent is tricked into calling the wrong tool, or the right tool with the wrong arguments.

Identity and privilege abuse

The agent is given or induced to use more authority than the workflow really requires.

Indirect prompt injection

Malicious instructions are embedded in documents, tickets, or other inputs that shape agent behavior.

Unsafe output handling

Bad output becomes a bad action when another system trusts it or executes it without review.

Prompt filtering is not enough by itself.

Prompt / context

Useful for shaping behavior, but too weak to be the final safety boundary.

Tool / server

The right place to enforce read/write boundaries, authorization, and denial.

Identity / policy

Controls who can act, what they can see, and which actions are never allowed.

If a lower layer can stop the action, the lower layer should stop the action.

Use OWASP-style categories to think in terms of real abuse patterns.

- Treat the agent as something that can be hijacked through context.
- Treat tool access as a real security boundary, not a convenience.
- Treat identity and privilege as part of the attack surface.
- Treat auditability as a control, not just a logging preference.

```
Threat-model questions

- What can be influenced?
- What can be called?
- What can be read?
- What can be written?
- What would prove the action was authorized?
```

No CLI demo today.

- Present a short incident trace with an unauthorized API call.
- Ask what action happened, not just what text looked suspicious.
- Identify where the malicious input entered the system.
- Map the breach to the missing control layer.

Tabletop outputs

Ingress point, action taken, likely abuse pattern, control that should have stopped it.

What success looks like

The group ends with controls and ownership, not just a description of the incident.

Security ownership must be explicit.

Tool owner

Defines what the tool is allowed to do and where the final deny decision lives.

Platform owner

Controls the execution environment, logging, and shared guardrails.

Application owner

Owns the workflow boundary and the business impact of a bad action.

Security reviewer

Challenges assumptions about permissions, exfiltration paths, and unsafe trust.

The security problem changes once the agent can act.

- Bad output is a quality issue.
- Bad action is a governance issue.
- The right control is usually lower than the prompt.
- Module 4 will make governance and auditability the main topic.

Core message

Prompt policy helps. Execution-layer controls decide whether the enterprise survives the breach.